## Module 01: LLM, Generative AI, and LangChain Basics

### 1. Large Language Models (LLMs)

LLM stands for Large Language Model. These models are trained on large amounts of text data to learn language patterns and generate human-like responses.

#### How LLMs Work

1. The input text is tokenized into smaller units called tokens.
2. Tokens are converted into numerical representations called embeddings.
3. The model processes the input and learns contextual relationships between tokens.
4. Autoregressive LLMs predict the next token based on the preceding context.
5. The process repeats until the response is generated.

**Examples:** GPT and Gemini.

### 2. Generative AI (GenAI)

Generative AI refers to AI systems that generate new content, such as text, images, audio, video, and code.

We can use existing pretrained models through APIs instead of building every model from scratch. Models can also be fine-tuned or trained when required.

### 3. API Keys

API keys are used to authenticate requests to AI services.

Different AI providers may have different API formats, parameters, and response structures.

**Important:** API keys should be kept secret and stored securely in environment variables rather than hardcoded into source code.

## 4. LangChain and LangGraph

### LangChain

LangChain is a framework used to build LLM applications by integrating models, prompts, tools, retrieval, and workflows.

### LangGraph

LangGraph is a framework for building stateful, multi-step AI workflows using graphs. It supports conditional execution, loops, and agent workflows.

## 5. Main Components of LLM Applications

### A. Models

1. **Language Models:** Process and generate text.
2. **Embedding Models:** Convert text into numerical vector representations.
3. **Multimodal Models:** Process or generate multiple types of data, such as text, images, audio, and video.

### B. Prompts

A prompt is the input or instruction provided to a model.

Common prompt types:

- **Simple Prompt:** A basic instruction or question.
- **System + User Prompt:** The system message defines general behavior, while the user message contains the user's request.
- **Prompt Template:** A reusable prompt containing variables.
- **Structured Output:** Constrains the response to a specified format, such as JSON.

A clear and specific prompt generally improves the usefulness of the output.

### C. Chains

Chains connect multiple operations into a workflow.

Example:

User Question → Prompt Template → LLM → Output

### D. Memory

Memory refers to mechanisms that preserve relevant conversation history or application state so the system can use previous information.

Memory implementation depends on the framework and application architecture.

### E. Indexes

Indexes organize external data so relevant information can be searched and retrieved efficiently.

They are commonly used in document retrieval and RAG applications.

### F. Agents

AI agents use a model, instructions, and available tools to decide what action to take next.

Example:

User Question → Agent → Search Tool → Retrieved Information → Final Answer

## 6. Retrieval-Augmented Generation (RAG)

RAG stands for Retrieval-Augmented Generation.

It allows an LLM application to retrieve relevant information from external sources and use that information when generating an answer.

### Why RAG Is Needed

An LLM may not know private documents, internal company information, or recently updated data. RAG can provide relevant external information as context.

### Basic RAG Workflow

1. Load external documents, such as PDFs.
2. Split documents into smaller chunks.
3. Generate embeddings for the chunks.
4. Store the vectors in a vector database or another retrieval index.
5. Convert the user's question into a suitable search representation.
6. Retrieve relevant chunks.

7. Provide the retrieved context and question to the LLM.
8. Generate an answer grounded in the retrieved information.

**Important:** RAG is one way to connect LLMs with external information. Tools, APIs, and other integrations can also provide external data.

## 7. Summary

- **LLM:** Understands context and generates language.
- **GenAI:** Generates new content.
- **API:** Provides programmatic access to a model or service.
- **LangChain:** Helps integrate components for LLM applications.
- **LangGraph:** Helps build stateful and multi-step workflows.
- **Prompt:** Provides instructions or input to a model.
- **Chain:** Connects multiple operations.
- **Memory:** Preserves relevant history or state.
- **Indexes:** Organize information for retrieval.
- **RAG:** Retrieves external information to provide context to an LLM.
- **Agent:** Selects and executes actions using available tools.

## 8. Practical Learning Order

1. Make an LLM API call.
2. Use system and user prompts.
3. Create reusable prompt templates.
4. Generate structured JSON output.
5. Build chains using LangChain.
6. Build a PDF question-answering application using RAG.
7. Build a stateful workflow or tool-using agent with LangGraph.